

Módulo 3 · Testing, Debugging y Documentación con IA

Validador de Contraseñas con IA — Ejercicio Práctico

Nombre del alumno/a	ANGEL ECHENIQUE
Fecha	23/03/2026
🔗 Enlace GitHub	

📌 **Instrucciones:** Responde cada bloque aplicando lo visto en clase. Puedes consultar tus apuntes y usar la IA para generar los tests. ¡No hay que escribir código desde cero, solo demostrar que entendiste cómo funciona el proceso!

BLOQUE 1 — Entiende la función antes de testearla (1.1 + 1.2)

Antes de pedir ningún test a la IA, hay que leer y entender bien qué hace el código. Aquí tienes una función que valida si una contraseña cumple las reglas de seguridad de una aplicación web:

```
def validar_contraseña(contraseña):
    """
    Valida que una contraseña cumpla los requisitos de seguridad.
    Reglas:
    - Longitud entre 8 y 20 caracteres
    - Al menos una letra mayúscula
    - Al menos un número
    - Al menos un carácter especial: !@#$%^&*()
    - Sin espacios en blanco
    Devuelve True si es válida, False si no lo es.
    """
    import re
    if not (8 <=
len(contraseña) <= 20):
        return False
    if not re.search(r'[A-Z]',
contraseña):
        return False
    if not re.search(r'[0-9]',
contraseña):
        return False
    if not re.search(r'[!@#$%^&*()]',
contraseña):
        return False
    if ' ' in
contraseña:
        return False
    return True
```

1.1 — Analiza la función

Responde con tus propias palabras antes de usar ninguna IA:

Pregunta sobre la función	Tu respuesta
¿Qué tipo de dato recibe la función como entrada?	Un string.
¿Qué devuelve si la contraseña es válida?	True.
¿Qué devuelve si la contraseña no cumple alguna regla?	False.
¿Cuántas reglas distintas valida la función?	1. Longitud entre 8 y 20 2. Al menos una mayúscula 3. Al menos un número 4. Al menos un carácter especial 5. Sin espacios

¿Qué pasa si el usuario envía una contraseña vacía ("")?	False.
--	--------

1.2 — Distingue pruebas unitarias de integración

Para la función `validar_contraseña()`, marca U (Unitaria) o I (Integración) y explica por qué:

U / I	Qué se prueba	¿Por qué es unitaria o de integración?
U	La función <code>validar_contraseña()</code> con la contraseña "Hola1!"	Porque podríamos solo testear la función aislada.
I	El flujo completo de registro: el usuario escribe una contraseña en el formulario web, se envía al servidor, pasa por <code>validar_contraseña()</code> y se guarda en la base de datos	Porque se comparten datos entre múltiples ellos. Se validan se autentifican se guardan.
U	La función devuelve False cuando la contraseña no tiene mayúsculas	Función que podríamos testear aislada. No conectamos datos en cadenas.
I	El endpoint <code>POST /registro</code> que llama a <code>validar_contraseña()</code> y responde con un JSON	Flujo de integración Porque involucra múltiples componentes (API, validación, respuesta JSON)

BLOQUE 2 — Frameworks de testing y comparativa (1.3)

Vamos a usar `pytest` para este ejercicio. Antes de escribir el prompt, repasa los frameworks:

Framework	Lenguaje	¿Para qué sirve principalmente?	¿Cuándo lo usarías?
pytest	Python	Tests unitarios y profesionales	Como herramienta de testeo unitario y integración.
unittest	Python	Es unit testing	Para testing unitario en Python
Jest	JavaScript	Testing de frontend/backend JS	Para Testing de frontend/backend JS
JUnit	Java	Testing en Java	Para el backend

2.2 — pytest vs unittest: ¿cuál elegirías?

Para este ejercicio con `validar_contrasena()`, marca cuál es la afirmación correcta sobre cada framework y justifica brevemente:

V / F	Afirmación	Justificación (solo si es Falsa)
V	Con pytest puedo escribir <code>assert validar_contrasena("Hola1!") == True</code> sin heredar ninguna clase.	
F	unittest es más fácil de empezar para principiantes porque necesita menos configuración.	Necesitas mas "boilerplate" que pytest para definir un test.
F	Si el proyecto ya usa unittest, no puedo ejecutar esos tests con pytest.	Es compatible sin reescribir el código.
V	pytest permite agrupar muchos casos de prueba en una sola función con <code>@pytest.mark.parametrize</code> .	
V	Para un proyecto nuevo y sin restricciones, pytest suele ser la elección más práctica.	

BLOQUE 3 — Construye el prompt profesional (1.4)

Recuerda que un prompt profesional para testing incluye 6 elementos clave. Vamos a construirlo para nuestra función `validar_contrasena()`:

Recuerda: Cuanto más contexto de negocio incluyas (¿para qué app es? ¿qué tipo de usuarios?), más útiles y específicos serán los tests que genere la IA.

3.1 — Rellena los 6 elementos del prompt

El primero ya está hecho como ejemplo. Completa el resto:

Nº	Elemento del prompt	¿Para qué sirve?	Tu ejemplo para <code>validar_contrasena()</code>
1	Código de la función	Le dice a la IA exactamente qué tiene que testear	python\nimport re\n\ndef validar_contrasena(contrasena):\n if not (8 <= len(contrasena) <= 20):\n return False\n if not re.search(r'[A-Z]', contrasena):\n return False\n if not re.search(r'[0-9]', contrasena):\n return False\n if not re.search(r'[@#\$%^&*()'],

			contrasena):\n return False\n if ' ' in contrasena:\n return False\n return True\n
2	Framework	Define cómo deben generarse los tests	pytest
3	Contexto de negocio	Añade realismo y requisitos específicos	Sistema de autenticación de usuarios en una aplicación bancaria
4	Casos a cubrir	Indica qué tipos de tests generar	Casos válidos, inválidos, límites, caracteres especiales, espacios, combinaciones incorrectas
5	Formato de salida	Hace que la respuesta sea directamente usable	Tests parametrizados con @pytest.mark.parametrize
6	Restricciones	Evita resultados incorrectos o poco útiles	No usar librerías externas, seguir estilo AAA, añadir explicación breve por caso

3.2 — El prompt completo

Escribe en el recuadro de abajo el prompt completo y profesional que le enviarías a la IA. Usa los 6 elementos de arriba y sé específico/a:

Actúa como un ingeniero de QA especializado en testing automatizado en Python usando pytest.
A continuación tienes una función que valida contraseñas: mandaria la funcion en cuestion.
Le daria un contexto: Esta función pertenece a un sistema de autenticación de una aplicación bancaria, donde la seguridad es crítica.
Genera tests cubriendo: - Casos válidos - Casos inválidos (sin mayúsculas, sin números, sin caracteres especiales, con espacios, etc.) - Casos límite (longitud 8 y 20) - Casos con caracteres especiales válidos e inválidos - Casos con caracteres inesperados (tildes, unicode)
Formato de salida: - Usa pytest - Usa @pytest.mark.parametrize - Incluye input y resultado esperado - Añade una breve explicación por cada caso
Restricciones: - No uses librerías externas adicionales

- Mantén estructura clara (AAA)
- No generes código innecesario

3.3 — El impacto del contexto de negocio

Vimos que añadir contexto de negocio cambia radicalmente los tests. Completa la columna derecha pensando que validar_contrasena() es para una app bancaria:

Aspecto del test	Sin contexto de negocio	Con contexto bancario
¿Exige contraseñas más largas?	No (min. 8 chars)	Si, podría pedirse una de mayor complejidad y longitud.
¿Se prueban contraseñas de directivos?	No	Claro, roles critico y seguridad reforzada.
¿Se registra el intento fallido?	No	si Auditorias y seguridad.
Número de casos de prueba generados	~10	Muchos mas de de 50.
¿Se prueban contraseñas con tildes (á)?	No	Si robustez.

BLOQUE 4 — Analiza y clasifica los tests generados (1.4 + 1.6)

La IA ha generado los siguientes tests. Léelos con atención y responde las preguntas:

```
import pytest from auth import
validar_contrasena

# Test 1 def
test_contrasena_valida():
    # Arrange    contrasena =
    "Segura1!"
    # Act    resultado =
    validar_contrasena(contrasena)
    # Assert
    assert resultado == True

# Test 2 def
test_sin_mayuscula():
    assert validar_contrasena("segura1!") == False
```

```
# Test 3 def
test_sin_numero():
    assert validar_contrasena("Seguridad!") == False

# Test 4 def
test_exactamente_8_caracteres():
    assert validar_contrasena("Segur1!a") == True

# Test 5 def
test_con_espacio():
    assert validar_contrasena("Segura 1!") == False
```

4.1 — Clasifica cada test

Para cada test, indica su tipo (N = Normal, L = Límite, E = Error) y si sigue el patrón AAA:

Test	¿Qué comprueba exactamente?	Tipo (N/L/E)	¿Sigue AAA? ¿Qué le falta?
Test 1	Contraseña completamente válida que cumple todas las reglas	N	✓ Sí sigue AAA (Arrange, Act, Assert claros)
Test 2	Contraseña sin mayúsculas → debe fallar	E	✗ No completo (no hay Arrange/Act explícitos, solo assert)
Test 3	Contraseña sin número → debe fallar	E	✗ No completo (falta estructura AAA clara)
Test 4	Contraseña con longitud mínima (8 caracteres) válida	L	✗ No completo (solo assert, sin separación AAA)
Test 5	Contraseña con espacios → debe fallar	E	✗ No completo (no sigue AAA completo)

4.2 — Detecta lo que falta

La IA no lo ha generado todo. Indica qué casos importantes faltan y clasifícalos:

Tipo (N/L/E)	Caso que falta	¿Por qué es importante probarlo?
L	Contraseña de 20 caracteres (máximo permitido).	Verifica el límite superior (errores típicos de validación).
E	Contraseña vacía ("").	Caso extremo muy común en validaciones.
E	Contraseña con caracteres no permitidos (ej: emojis o unicode raro).	Robustez del sistema ante inputs inesperados.
L	Contraseña de 7 caracteres.	Comprueba que justo por debajo del mínimo falla.

BLOQUE 5 — Buenas prácticas y errores comunes (1.7)

Un equipo está usando la IA para generar todos los tests de un sistema de login. Marca BP (Buena Práctica) o EC (Error Común) y, para los errores, escribe cómo corregirlos:

BP / EC	Afirmación relacionada con <code>validar_contrasena()</code>	Corrección (solo si es EC)
BP	Ejecutar cada test generado manualmente para comprobar que pasan y que el mensaje de error es claro.	—
EC	Asumir que si la IA generó 10 tests, ya está todo cubierto y no hace falta pensar más casos.	Añadir más casos manualmente (edge cases, límites, inputs inválidos) y revisar cobertura real.
BP	Añadir a mano el caso <code>validar_contrasena("")</code> porque la IA no siempre lo incluye.	—
EC	Pedir en un solo prompt todos los tests de todos los módulos del sistema (login, registro, perfil, pagos).	Dividir en prompts pequeños por módulo o funcionalidad para obtener tests más precisos y mantenibles.

BP	Documentar en el código que los tests fueron generados con IA y la fecha en que se revisaron.	—
EC	Usar directamente los mocks que generó la IA sin comprobar si simulan bien el comportamiento real.	Validar los mocks contra el comportamiento real (API o lógica) y ajustarlos manualmente.

BLOQUE 6 — Caso real — Síntesis

▢ **Caso real:** Una startup fintech lanza su app de pagos. El CTO propone: "La IA ya generó 50 tests para el módulo de autenticación (incluye validar_contrasena()). Los desplegamos directamente en producción esta tarde para cumplir el plazo."

6.1 — ¿Qué riesgos tiene esta decisión?

Menciona al menos 3 riesgos concretos relacionando cada uno con algún bloque de este ejercicio:

Falta de cobertura de casos críticos (Bloque 4 — casos faltantes) La IA suele omitir edge cases (vacíos, límites superiores, inputs raros). Riesgo: fallos en situaciones reales no contempladas.
Uso incorrecto de mocks o comportamiento no realista (Bloque 5 — errores comunes) Los mocks generados pueden no representar la lógica real. Riesgo: los tests validan algo que no ocurre en producción.
Falta de contexto de negocio (Bloque 3 — prompt profesional) Si el prompt no incluía contexto fintech/bancario, los tests serán genéricos. Riesgo: no se validan requisitos de seguridad críticos.
No validación humana (Bloque 5 — buenas prácticas) Confiar directamente en la IA sin revisión. Riesgo: errores graves pasan a producción

6.2 — ¿Cómo lo harías tú de forma responsable?

Momento	Qué harías (pasos concretos)
Antes de escribir el prompt a la IA	Analizar la función (validar_contrasena), identificar reglas, límites y posibles edge cases. Definir requisitos de negocio (seguridad fintech).

Al escribir el prompt	Incluir los 6 elementos: código, framework (pytest), contexto bancario, tipos de casos (V, I, L, edge), formato (parametrize), restricciones claras.
Después de recibir los tests	Revisar manualmente cada test, validar lógica, ejecutar tests, detectar falsos positivos y añadir casos faltantes (vacíos, límites superiores, unicode, etc.).
Antes de desplegar a producción	Ejecutar suite completa, medir cobertura (ej: pytest-cov), revisar resultados, hacer validación QA, posiblemente code review y pruebas de integración.